

Exercícios : qual é a saída?

Eduardo Barella 24088000-2

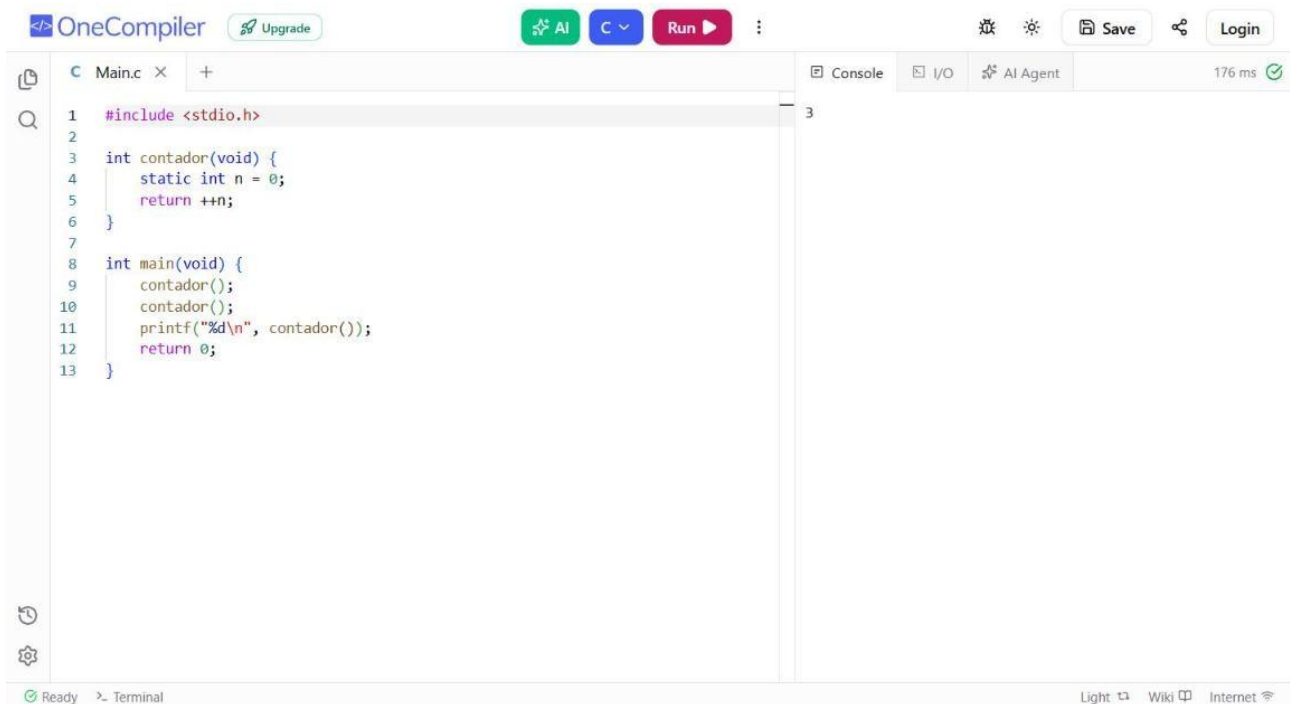
Matheus Mascarello 24120740-2

Exercício 1/3

```
int contador(void) {  
    static int n = 0;  
    return ++n; } // em  
main: contador();  
    contador();  
    printf("%d\n", contador());
```

(a) Saída: 3

(b) **Conceito:** variável local **estática** (Sebesta §9.4, p. 373-375). O `static int n = 0` é inicializado uma única vez e mantém o valor entre as chamadas: 1ª chamada → 1, 2ª → 2, 3ª → 3, que é o valor impresso. Se `n` fosse uma variável dinâmica da pilha (sem `static`), cada chamada começaria em 0 e a saída seria 1.



The screenshot shows the OneCompiler web IDE. The code editor contains the following C code:

```
1 #include <stdio.h>  
2  
3 int contador(void) {  
4     static int n = 0;  
5     return ++n;  
6 }  
7  
8 int main(void) {  
9     contador();  
10    contador();  
11    printf("%d\n", contador());  
12    return 0;  
13 }
```

The console output on the right shows the number 3. The status bar at the bottom indicates 'Ready' and 'Terminal'.

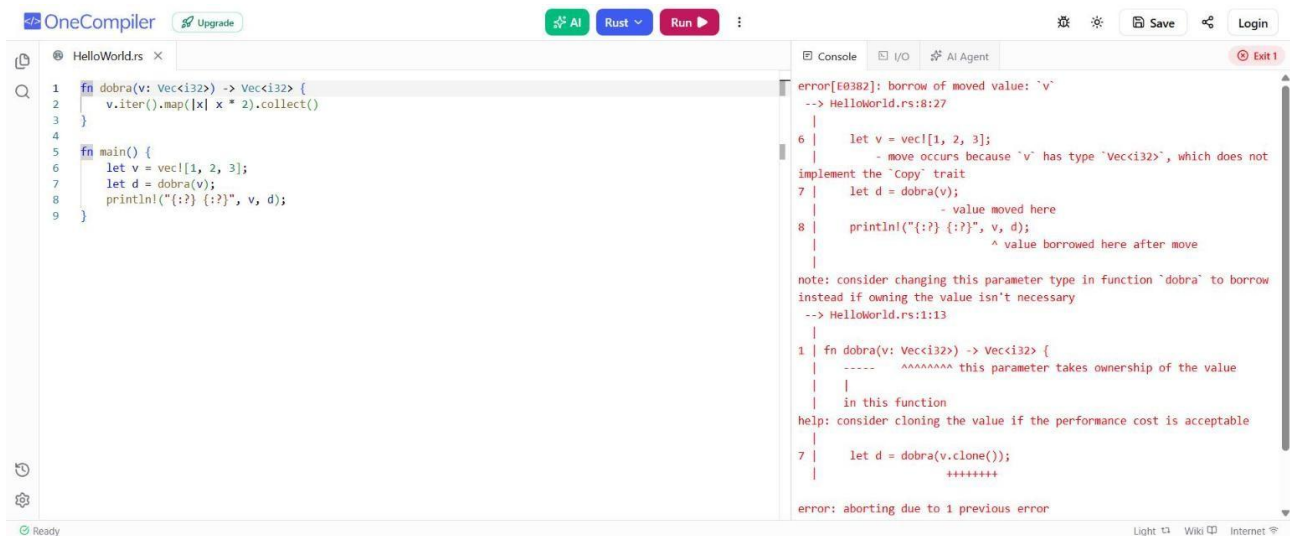
Exercício 2/3 - Rust

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
} let v = vec![1, 2, 3]; let d =
dobra(v); println!("{:?} {:?}", v,
d);
```

(a) **Saída:** não há saída, porque o programa **não compila**: error[E0382]: borrow of moved value: `v`

(b) **Conceito:** passagem por **posse (move)**. Como `Vec<i32>` não implementa `Copy`, a chamada `dobra(v)` transfere a posse do vetor para a função. Depois disso, `v` não pode mais ser usado em `main`, e o compilador barra o `println!`. Para corrigir, a função pode receber uma referência: `fn`

`dobra(v: &[i32])`, chamada com `dobra(&v)`. Assim a saída fica `[1, 2, 3] [2, 4, 6]`.



Exercício 3/3 - Python

```
total = 0
def adiciona(x):
    total = total + x
    return total
print(adiciona(5))
```

(a) **Saída:** erro em tempo de execução, `UnboundLocalError: cannot access local variable 'total' where it is not associated with a value`

(b) **Conceito:** ambiente de referenciamento local (Sebesta §9.4) e efeitos colaterais. Como há uma atribuição a `total` dentro da função, o Python considera `total` uma variável local em toda a função. Quando a expressão `total + x` é avaliada, essa variável local ainda não tem valor, e a global não é usada. Para corrigir, declare `global total` no início da função (a saída fica 5). A alternativa mais limpa é evitar o efeito colateral: `def adiciona(total, x): return total + x.`

The screenshot shows the OneCompiler Python IDE interface. The editor on the left contains the following Python code:

```
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
```

The right-hand console displays a traceback for an `UnboundLocalError`. The error message is: `UnboundLocalError: cannot access local variable 'total' where it is not associated with a value`. The traceback indicates the error occurred in the `adiciona` function at line 4, during the execution of `print(adiciona(5))` at line 7 of `main.py`.